

Відповіді на контрольні запитання

Тема 23: Шаблон проектування Відвідувач (Visitor)

23.1.1. Для чого застосовується шаблон проектування Відвідувач?

Шаблон **Відвідувач (Visitor)** застосовується у таких випадках:

- Коли вам потрібно виконати якусь операцію над усіма елементами складної структури об'єктів (наприклад, деревом).
- Коли ви хочете додати нову функціональність до існуючих класів, не змінюючи при цьому їхній код (дотримання принципу відкритості/закритості).
- Коли над класами об'єктів часто виконуються різні операції, і ви хочете уникнути «засмічення» класів цими операціями.
- Коли поведінка алгоритму залежить від класів об'єктів, над якими він працює, і ці класи рідко змінюються, але часто додаються нові алгоритми.

Основна мета: відокремити алгоритми від структур об'єктів, на яких вони базуються.

23.1.2. Наведіть основні етапи реалізації шаблону Відвідувач.

Реалізація шаблону Відвідувач складається з наступних кроків:

1. Створення інтерфейсу Відвідувача (Visitor): Оголошення набору методів «visit», по одному для кожного класу конкретних елементів, які існують у вашій програмі.
2. Створення інтерфейсу Елемента (Element): Додавання абстрактного методу «accept(Visitor v)» у базовий клас або інтерфейс ієрархії об'єктів.
3. Реалізація методу асерт у конкретних елементах: Кожен конкретний клас елемента повинен реалізувати метод асерт,

викликаючи відповідний йому метод відвідувача (наприклад: `v.visitConcreteElementA(this)`).

4. Створення конкретних відвідувачів: Реалізація класів-відвідувачів, кожен з яких описує певну операцію, що виконується над усіма елементами структури.
5. Запуск механізму: Клієнт створює об'єкт відвідувача та передає його в метод асерт елемента (або проходить циклом по колекції елементів).

23.1.3. Наведіть основні переваги та недоліки застосування шаблону Відвідувач.

Переваги:

- Принцип відкритості/закритості: дозволяє легко додавати нові операції над об'єктами різних класів.
- Принцип єдиної відповідальності: виносить споріднені алгоритми в один клас.
- Накопичення стану: відвідувач може збирати інформацію під час обходу структури об'єктів.

Недоліки:

- Складність при додаванні нових класів елементів: якщо додати новий тип елемента, доведеться оновити всі існуючі класи відвідувачів.
- Порушення інкапсуляції: відвідувачам може знадобитися доступ до приватних полів і методів елементів, що змушує робити їх публічними.